



Automate Repetitive Tasks with SaltStack

SaltStack is an open source, scalable, flexible systems management software that can be used for data centre automation, cloud orchestration, server provisioning and configuration management.



CM or configuration management is common terminology in the software industry. For those of us who do not know what it is, it's the process of automating sets of tasks that are repetitive, e.g., building a machine to match the company or group standards. Some of the prominent tools that help you to do this are SaltStack, Ansible, Puppet, Chef, CFEngine, Bcfg2, Quattor, Rundeck and Vagrant. We will concentrate on SaltStack.

A DevOps engineer is a new role that has evolved in the software industry. One of the key skills a DevOps engineer is expected to have, among many others, is automation skills. Whoever has read the four-part series of articles on Puppet at <http://www.opensourceforu.com/tag/puppet/> must surely know about configuration management by now, at least from a Puppet perspective. But don't worry if you don't know too much; one can always learn.

Why SaltStack?

There are a few reasons why we have chosen SaltStack as a configuration management tool. These are:

- Orchestration, predictability and automation
- The power available to DevOps
- It is open source (Apache licence), free, and has pretty wide community support in case of hiccups

- A full-fledged client/server model
- It is written entirely in Python
- Marks systems for easier management
- Uses the fast, powerful and light communication layer—ZeroMQ
- Uses the well established jinja2 platform for logic (uses a variety of languages to render the configuration files; yaml + jinja2 is the default template engine)
- Password-less root-based remote execution—runs predefined or custom commands on remote hosts
- It uses a push technology, where the onus is on the server to push configuration details to the client
- Completely encrypted communication and data transfer
- It is available on all UNIX/GNU/Linux platforms under the following pkg names:
 - salt-master
 - salt-minion

Glossary

This is already available at the SaltStack glossary page; I am just re-pasting a few terms that we will see more often.

SLS: The SaLT State File Extension.

Salt State: Consists of identifiers (resources to operate on), a state module (functionality to manage a type of resource) and function declarations (definition of a desired state of a resource), and sometimes function parameters (specific details for the state function).

Master: A central Salt daemon, from which commands can be issued to listening minions.

Minion: A server running a Salt minion daemon, which can listen to commands from a master and perform the requested tasks. Generally, minions are servers that are to be controlled using Salt.

Masterless: A minion that does not require a Salt master to operate. All configuration is local.

Top file: Determines which SLS files should be applied to various systems, and organises those groups of systems into environments.

High state: The collection of states to be applied to a system.

High data: Data structure in a SLS file that represents a